

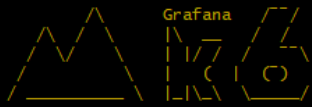
Napredni razvoj programske potpore za web - Dodatni projekt

Opis rješenja

Za izradu projekta korištene su sljedeće tehnologije: Node.js za backend aplikaciju, Docker i Docker Compose za kontejnerizaciju i orkestraciju, NGINX kao load balancer/reverse proxy, k6 za load testing te Python (NumPy, Matplotlib) za analizu rezultata i generiranje grafova. Arhitektura se sastoji od NGINX-a koji prima zahtjeve na portu 8080 i distribuira ih round-robin algoritmom na četiri Node.js instance. Svaka instanca ima /ping endpoint koji vraća "pong" s umjetnim kašnjenjem koje prati log-normalnu distribuciju - ovo simulira realistično ponašanje web servisa gdje većina zahtjeva bude brza, ali postoji "long tail" sporijih zahtjeva.

Node.js aplikacija koristi Box-Muller transformaciju za generiranje log-normalno distribuiranih vremena odgovora s podesivim parametrima ($\mu=4.5$, $\sigma=0.8$). Dodatno troši CPU resurse matematičkim operacijama kako bi simulirala realno opterećenje. Load testiranje provedeno je s k6 alatom koristeći 200 virtualnih korisnika kroz 5 minuta. Za automatizaciju testiranja korištene su bash skripte (run-scale-up.sh i run-scale-down.sh) koje pokreću k6 test i u pozadini postupno dodaju/gase Docker instance u definiranim intervalima. Za Scale Up test krenulo se s jednom instancom pa su postupno dodavane nove, dok je za Scale Down test bilo obrnuto - krenulo se sa sve četiri instance koje su postupno gašene. Rezultati su analizirani Python skriptom (analyze.py) koja generira percentil grafove (tail latency) i histograme distribucije vremena odgovora.

```
ivoga@IvoPC MINGW64 ~/Documents/Git local repositories/web2-sedmi-projekt (main)
$ ./run-scale-down.sh
=== SCALE DOWN TEST (4 -> 1) ===
Container web2-sedmi-projekt-node4-1 Running
Container web2-sedmi-projekt-node1-1 Running
Container web2-sedmi-projekt-node3-1 Running
Container web2-sedmi-projekt-node2-1 Running
Container web2-sedmi-projekt-nginx-1 Running
[0:00] Test started with 4 nodes
```



```

  execution: local
    script: load-test.js
    output: json (results/scale-down.json)

  scenarios: (100.00%) 1 scenario, 50 max VUs, 5m30s max duration (incl. graceful stop):
    * constant_load: 50 looping VUs for 5m0s (gracefulStop: 30s)

[1:15] Stopping node4...
Container web2-sedmi-projekt-node4-1 Stopping
Container web2-sedmi-projekt-node4-1 Stopped
running (1m25.6s), 50/50 VUs, 18615 complete and 0 interrupted iterations
constant_load [=====>-----] 50 VUs  1m25.6s/5m0s
```

Pokretanje projekta

Preduvjeti:

- Docker i Docker Compose
- Grafana k6
- Python s numpy i matplotlib bibliotekama (pip install numpy matplotlib)

Build Docker images:

docker-compose build

Scale Up Test:

Skripta run-scale-up.sh automatski pokreće NGINX i jednu Node.js instancu, zatim pokreće k6 test (200 VU, 5 minuta) i svakih 75 sekundi dodaje novu instancu.

Naredba: ./run-scale-up.sh

Vremenski raspored:

[0:00] Test kreće s 1 instancom

[1:15] Dodaje se node2

[2:30] Dodaje se node3

[3:45] Dodaje se node4

[5:00] Test završava

Scale Down Test:

Skripta run-scale-down.sh automatski pokreće NGINX i sve četiri Node.js instance, zatim pokreće k6 test (200 VU, 5 minuta) i svakih 75 sekundi gasi jednu instancu.

Naredba: ./run-scale-down.sh

Vremenski raspored:

[0:00] Test kreće s 4 instance

[1:15] Gasi se node4

[2:30] Gasi se node3

[3:45] Gasi se node2

[5:00] Test završava (ostaje samo node1)

Generiranje grafova:

python analyze.py results/scale-up.json "Scale Up"

python analyze.py results/scale-down.json "Scale Down"

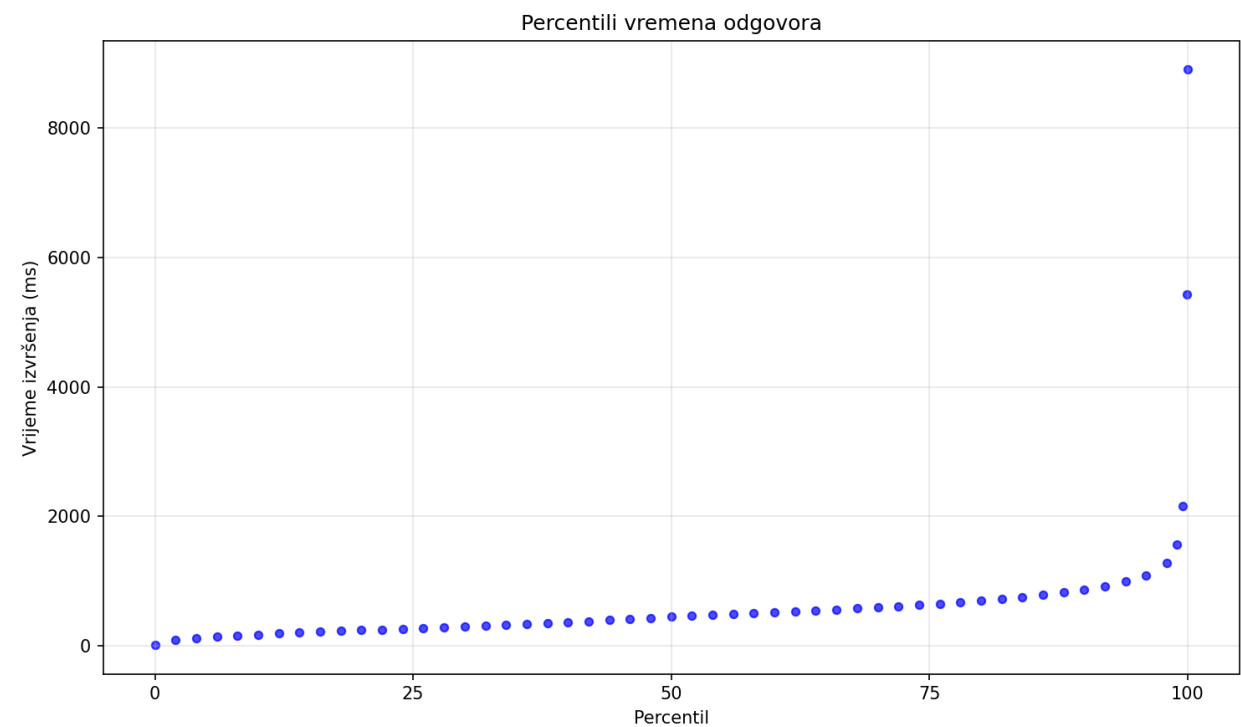
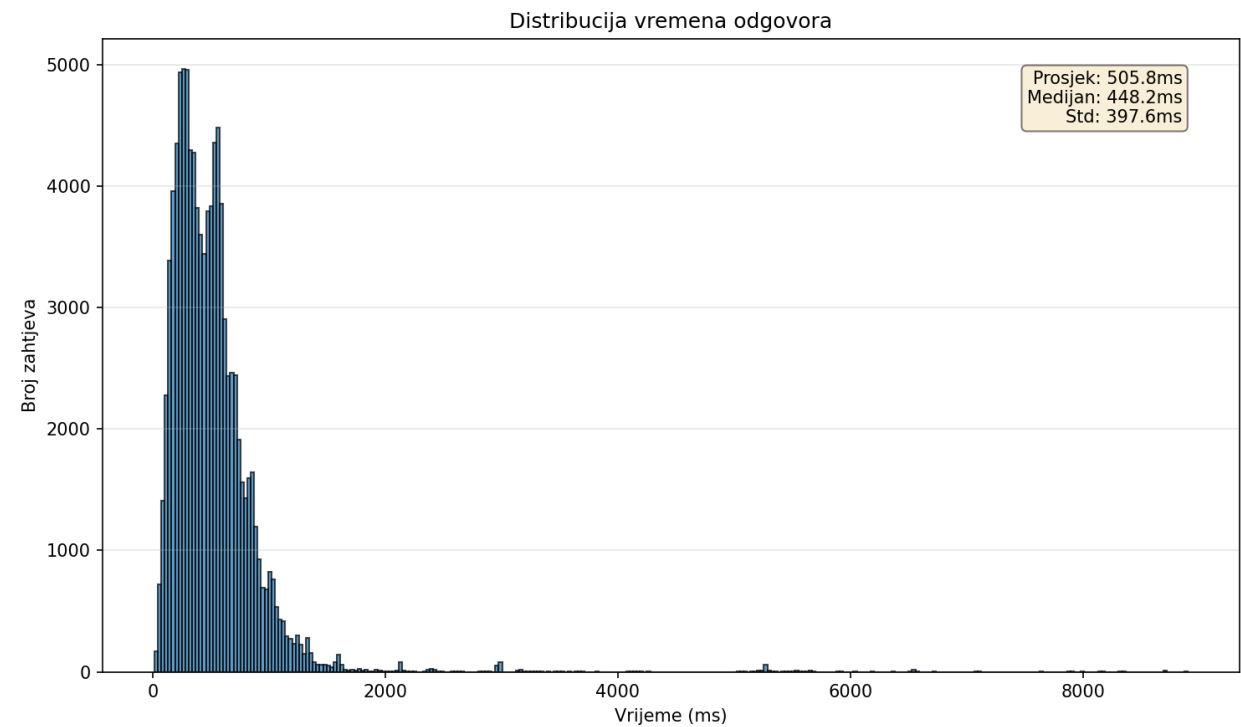
Skripta generira četiri slike:

- scale-up-percentiles.png
- scale-up-histogram.png
- scale-down-percentiles.png
- scale-down-histogram.png

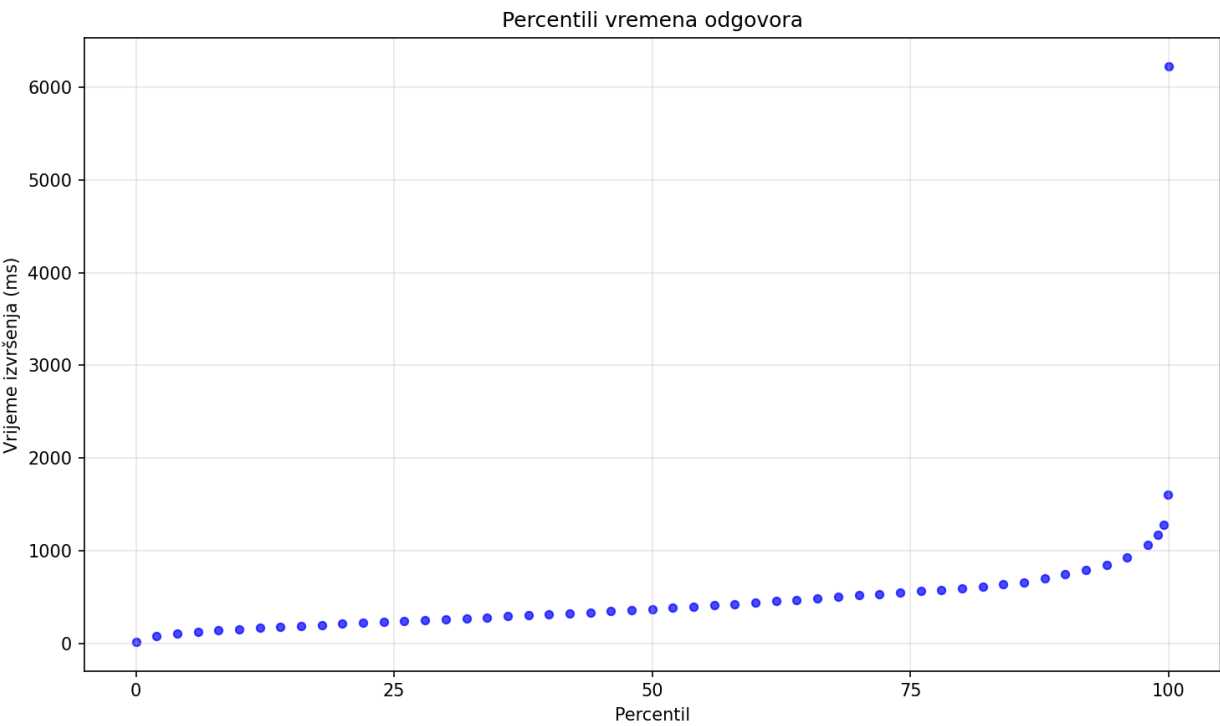
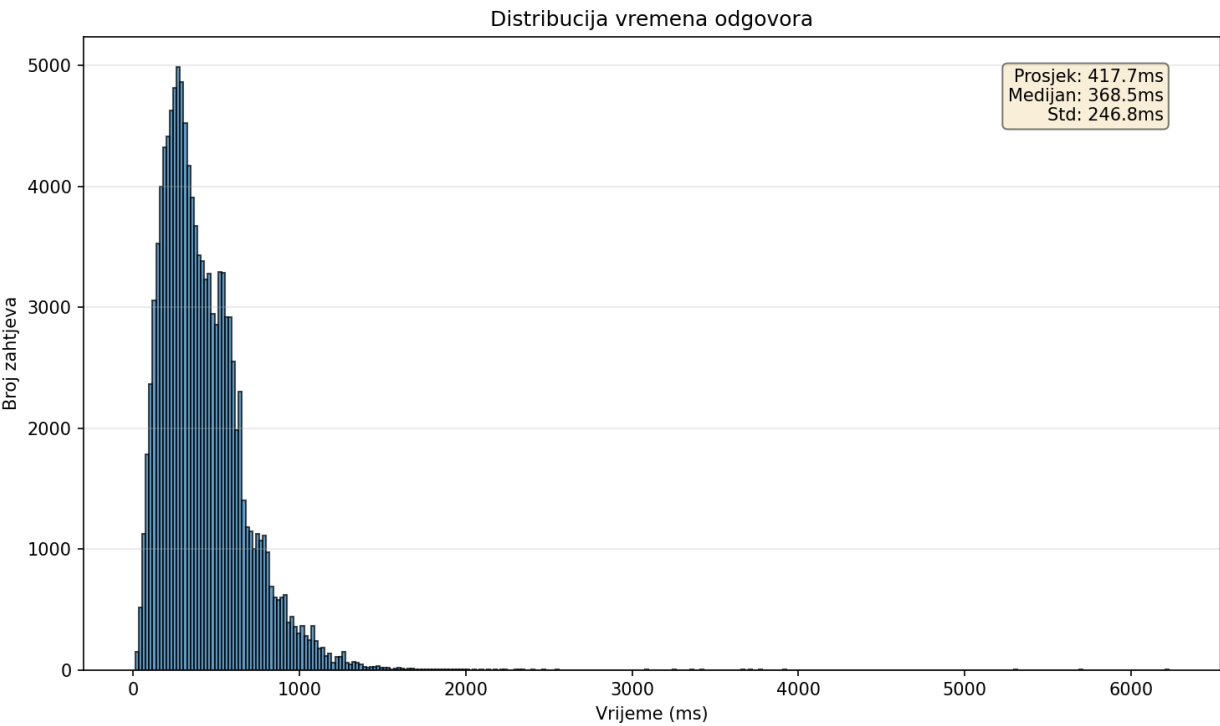
Rezultati

	Ukupno zahtjeva	Prosjek(ms)	Medijan(ms)	p90(ms)	p95(ms)	p99(ms)	Max(ms)
Scale Up	99022	505.84	448.17	868.84	1036.59	1567.46	8899.05
Scale Down	115888	417.71	368.46	746.51	883.71	1171.78	6226.30

Scale up



Scale down



Komentar rezultata i zaključak

Scale Down scenarij je obradio 17% više zahtjeva uz niže vrijeme odgovora u svim slučajevima iz tablice. Prosječno vrijeme odgovora bilo je niže za 88 ms, a najveća razlika vidljiva je u ekstremnim vrijednostima - maksimalno vrijeme u Scale Up scenariju bilo je čak 43% veće nego u Scale Down scenariju.

Oba histograma potvrđuju log-normalnu distribuciju vremena odgovora. Vidljiv je glavni vrh u području 200-400 ms, nakon čega slijedi asimetrični "dugi rep".

Scale Up histogram ima veću standardnu devijaciju (397.6 ms) što znači veću varijabilnost vremena odgovora. Rep distribucije je širi i dulji, s vidljivim zahtjevima čak iznad 8000 ms. Na grafu su također vidljivi mali izolirani stupci u području visokih kašnjenja (iznad 4000 ms) koji predstavljaju zahtjeve koji su "zapeli" tijekom zagušenja sustava. To pokazuje da je sustav u početnoj fazi testa, kada je imao samo jednu instancu, doživio značajna zagušenja.

Scale Down histogram ima nižu standardnu devijaciju (246.8 ms) i užu distribuciju fokusiranu oko medijana. Za razliku od Scale Up histograma, ovdje nema izoliranih stupaca u području ekstremnih vrijednosti - distribucija je kompaktnija i s manje outliera.

Grafovi percentila pokazuju kako se vrijeme odgovora povećava za najsporije zahtjeve. U oba scenarija vidljiv je stabilan rast do 90. percentila, nakon čega dolazi do strmijeg skoka karakterističnog za sustave s čekanjem u redu.

Ključna razlika vidljiva je na p99 razini - vrijeme koje doživljava 1% najsporijih korisnika. U Scale Up scenariju p99 iznosi 1567 ms, dok u Scale Down scenariju iznosi 1172 ms. Ova razlika od oko 400 ms pokazuje da horizontalno skaliranje najviše pomaže upravo najsporijim zahtjevima.

Rezultati pokazuju da je sustav osjetljiviji na pokretanje s nedovoljno resursa (Scale Up) nego na postupno smanjenje kapaciteta (Scale Down). Kada sustav kreće s jednom instancom pod punim opterećenjem, dolazi do zagušenja koje rezultira visokim kašnjenjima i velikim brojem sporih zahtjeva. S druge strane, kada sustav kreće s više instanci i postupno ih gasi, performanse ostaju stabilnije.